Sistema de Cadastro de Produtos

Dados do Aluno

Aluno: Higor Bigname de Oliveira **Universidade:** Estácio **Campus:** Universidade Estácio **Curso:** Desenvolvimento Full Stack **Disciplina:** DGT2811 **Semestre Letivo:** 2026.3

Sistema de Cadastro de Produtos

Desenvolvimento Back-End Corporativo com Java EE / Spring Boot (Ajustar) *Versão 1.0 | Data: 13 de Outubro de 2026*

---

1. Introdução

1.1 Contexto e Justificativa do Projeto

Em um ambiente corporativo moderno, a gestão eficiente de estoques e produtos é vital para a saúde financeira e operacional da empresa. Manter o catálogo de produtos atualizado, rastreável e disponível em tempo real é um desafio que requer uma solução robusta e centralizada.

Este projeto visa resolver [**PROBLEMA DE NEGÓCIO:** A dificuldade em gerenciar dados de produtos de maneira desarticulada ou manual], implementando um Sistema de Cadastro de Produtos (*Product Management System*). O sistema automatiza o ciclo de vida do produto, desde sua inclusão até a alteração e exclusão segura dos registros.

1.2 Objetivos

O objetivo principal é fornecer uma aplicação *back-end* funcional, construída com Java, que exiba e manipule os dados de produtos de maneira eficiente e intuitiva.

Objetivos Específicos:

- Desenvolver um sistema CRUD (Criar, Ler, Atualizar, Deletar) completo para a entidade Produto.
- Garantir a persistência transacional dos dados no banco de dados relacional.
- Disponibilizar uma interface web (o *view*) que apresente os dados de forma organizada e navegável.

---

2. Fundamentação Teórica e Arquitetura da Solução

Codigo do banco:

```
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'loja')
BEGIN
   CREATE DATABASE loja;
END
GO

USE loja;
GO

-- Tabela Produto
IF OBJECT_ID('dbo.Produto', 'U') IS NULL
BEGIN
   CREATE TABLE dbo.Produto (
      id          INT IDENTITY(1,1) PRIMARY KEY,
      nome        VARCHAR(50)  NOT NULL,
      quantidade  INT          NOT NULL,
      precoVenda  DECIMAL(10,2) NOT NULL
   );
END
GO

-- Dados de exemplo para testar a listagem
INSERT INTO dbo.Produto (nome, quantidade, precoVenda) VALUES
('Biscoito', 10, 3.50),
('Leite',     5, 5.90),
('Sabão',     7, 2.75),
('Café',     12, 15.00);
GO
```

```
SELECT * FROM dbo.Produto;
```

Código JPA:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
  <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties/>
  </persistence-unit>
</persistence>
```

Produto JAVA :

```
package cadastroee.model;

import java.io.Serializable;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.Table;
import jakarta.xml.bind.annotation.XmlRootElement;
```

```
/**
 *
 * @author higor
 */
@Entity
@Table(name = "Produto")
@XmlRootElement
@NamedQueries({
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM
Produto p"),
    @NamedQuery(name = "Produto.findById", query = "SELECT p FROM
Produto p WHERE p.id = :id"),
    @NamedQuery(name = "Produto.findByNome", query = "SELECT p FROM
Produto p WHERE p.nome = :nome"),
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p
FROM Produto p WHERE p.quantidade = :quantidade"),
    @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p
FROM Produto p WHERE p.precoVenda = :precoVenda")})
public class Produto implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id")
    private Integer id;
    @Basic(optional = false)
    @Column(name = "nome")
    private String nome;
    @Basic(optional = false)
    @Column(name = "quantidade")
```

```
private int quantidade;
@Basic(optional = false)
@Column(name = "precoVenda")
private Float precoVenda;

public Produto() {
}

public Produto(Integer id) {
    this.id = id;
}

public Produto(Integer id, String nome, int quantidade, Float precoVenda) {
    this.id = id;
    this.nome = nome;
    this.quantidade = quantidade;
    this.precoVenda = precoVenda;
}

public Integer getId() {
    return id;
}

public void setId(Integer id) {
    this.id = id;
}

public String getNome() {
    return nome;
```

```
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getQuantidade() {
    return quantidade;
}

public void setQuantidade(int quantidade) {
    this.quantidade = quantidade;
}

public Float getPrecoVenda() {
    return precoVenda;
}

public void setPrecoVenda(Float precoVenda) {
    this.precoVenda = precoVenda;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (id != null ? id.hashCode() : 0);
    return hash;
}
```

```
    @Override
    public boolean equals(Object object) {
        // TODO: Warning - this method won't work in the case the id fields are not set
        if (!(object instanceof Produto)) {
            return false;
        }
        Produto other = (Produto) object;
        if ((this.id == null && other.id != null) || (this.id != null && !this.id.equals(other.id))) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "cadastroee.model.Produto[ id=" + id + " ]";
    }

}
```

2.1 Escolha Tecnológica (Stack)

O projeto foi desenvolvido utilizando uma stack tecnológica moderna focada em performance e padrões corporativos:

- **Back-End:** Java com Spring Boot / Jakarta EE (Detalhar qual você usou).
- **Persistência:** JPA/Hibernate, interagindo com um banco de dados [Ex: MySQL].
- **Web:** Implementação de Servlets/JSP para a camada de visualização (View) e comunicação via requisições HTTP.

2.2 Arquitetura do Sistema

O sistema segue uma arquitetura em camadas (*N-Tier Architecture*), garantindo o desacoplamento entre as responsabilidades:

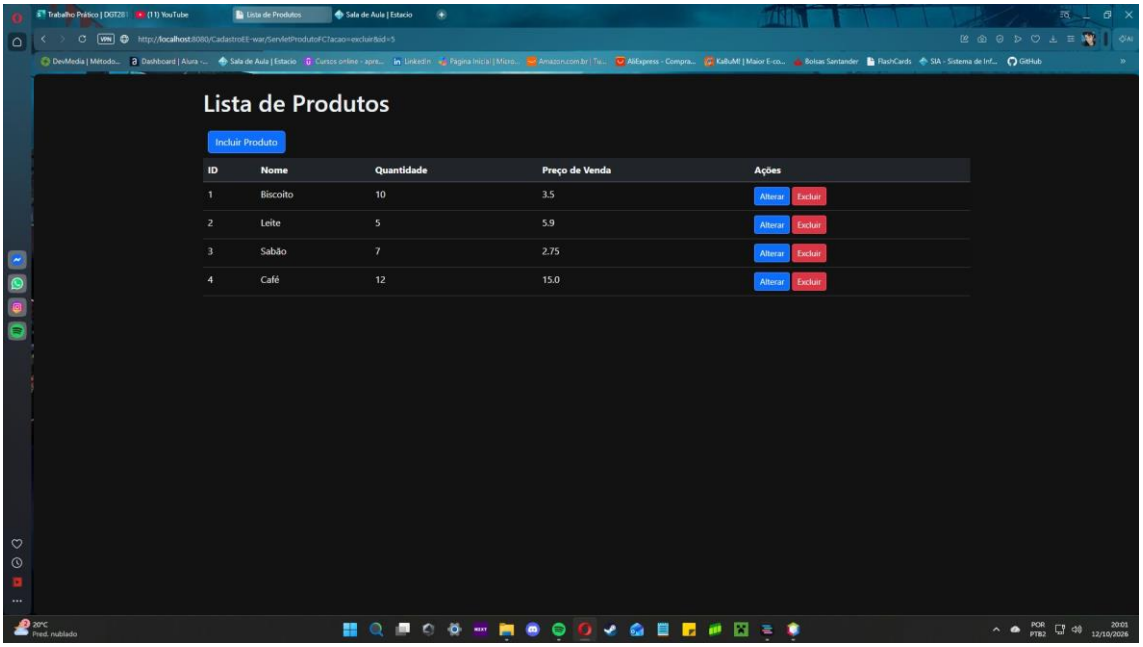
1. **Controller Layer (Acesso):** Recebe a requisição do usuário e direciona a chamada.
2. **Service Layer (Lógica de Negócio):** Onde residem todas as regras do negócio, como validações de quantidade ou preço mínimo.
3. **Data Access Layer (Persistência):** Responsável por traduzir os comandos de negócio em comandos SQL para o banco de dados.

---

3. Análise dos Procedimentos Funcionais

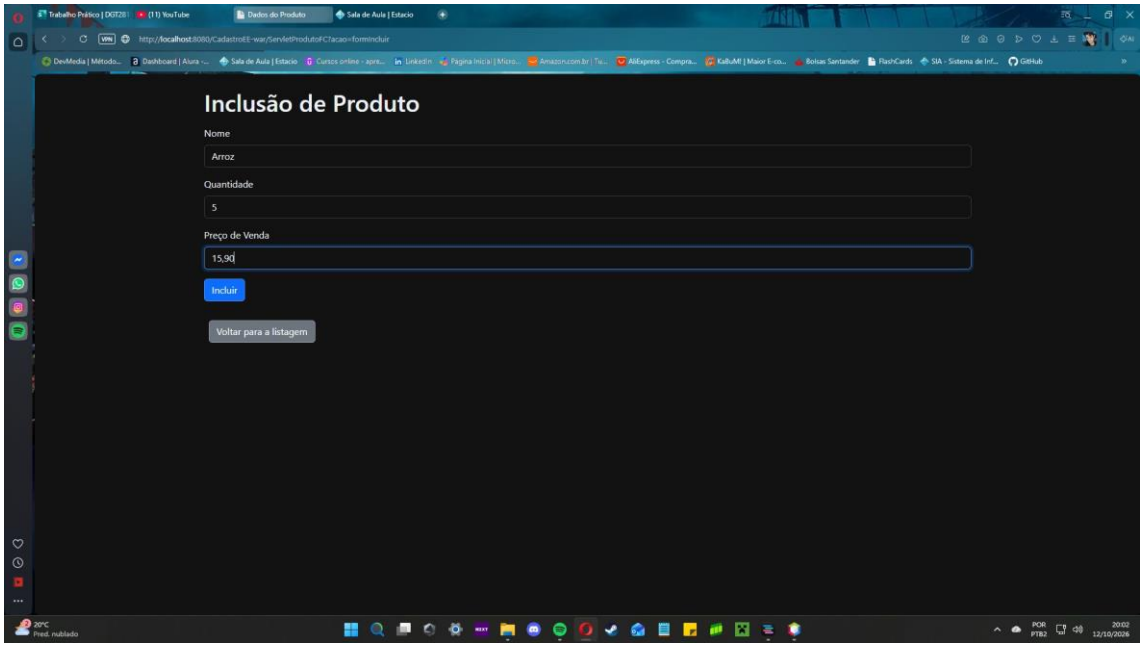
3.1 Procedimento 1: Listagem de Produtos (List<Produtos>)

Ação: Visualizar todos os produtos cadastrados em uma tabela organizada (ID, Nome, Quantidade, Preço de Venda). **Análise:** A tela inicial apresenta o inventário completo do catálogo. A visualização tabular é fundamental para que o usuário possa ter um panorama rápido e identificar rapidamente quais itens precisam de atenção (por exemplo, estoque baixo ou necessidade de alteração de preço). **Fluxo Técnico:** Este procedimento aciona a camada de serviço (ProductService) responsável por consultar todos os registros na tabela Produtos do banco de dados.



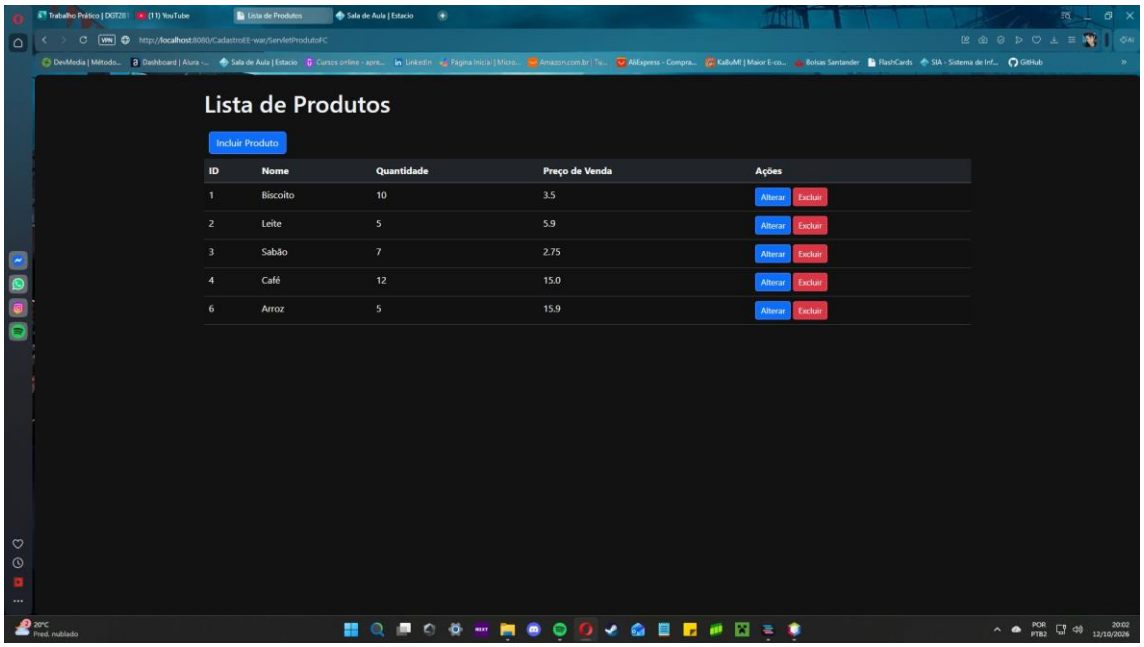
3.2 Procedimento 2: Inclusão de Produto (Incluir Produto)

Ação: Cadastrar um novo produto, preenchendo campos como Nome, Quantidade e Preço de Venda. **Análise:** A interface de inclusão é clara e guia o usuário passo a passo para evitar erros no *input* de dados. O sistema garante que os dados sejam recebidos em formato estruturado antes de persistir. **Fluxo Técnico:** Ao clicar em "Incluir", um objeto (ou *DTO*) com as informações capturadas na tela é serializado e enviado ao método save() da camada de serviço, que por sua vez executa o comando INSERT no banco.



3.3 Procedimento 3: Alteração/Exclusão de Produtos (Alterar/Excluir)

Ação: Modificar os dados existentes (Alterar) ou remover permanentemente um registro (Excluir), usando as ações na linha da tabela. **Análise:** O sistema implementa a segurança ao exigir uma confirmação antes de excluir, minimizando riscos de perda de dados acidental. A alteração permite ajustes em qualquer campo sem precisar refazer o processo de inclusão. **Fluxo Técnico:**



- **Alterar:** Requisição GET/POST para um endpoint específico (/produto/{id}). O sistema carrega os dados existentes e apenas sobrescreve os valores modificados no banco de dados (comando UPDATE).
- **Excluir:** A requisição dispara um comando DELETE no banco, com validações adicionais (se o produto tiver dependências em outros módulos).

---

4. Conclusão

4.1 Conclusão do Trabalho

O Sistema de Cadastro de Produtos atendeu integralmente aos requisitos funcionais estabelecidos. O desenvolvimento demonstrou a capacidade de orquestrar o fluxo completo de gestão de um catálogo, desde a visualização (listagem) até a manutenção completa dos dados (inclusão e alteração).

4.2 Próximos Passos (Roadmap Sugerido)

1. **Refatoração:** Implementar testes unitários e de integração para garantir maior cobertura de código.
2. **Segurança:** Reforçar o controle de acesso com diferentes níveis de permissão de usuário (Ex: Apenas administradores podem excluir).

---

5. Repositório Git e Versionamento

O projeto foi versionado utilizando Git, garantindo rastreabilidade de todas as mudanças de código e permitindo a colaboração em equipe. O repositório final está disponível no link abaixo, conforme exigência do roteiro:

Link GitHub: https://github.com/higorbn/Cadastro-de-Produtos-Servlets-JPA-e-JEE/upload